# An Exploration of Timing Attacks

Elena Rawlinson

Spring 2026

## 1 Introduction

Every time you send a text, open a web browser, or get an email, your data is kept secure using cryptographic protocols. For example, when you type a website name into your browser you see the text "https," that stands for hypertext transfer protocol secure, which is a protocol for transferring encrypted data between your computer and a server. Sending un-encrypted information across the internet is the modern day equivalent of yelling to someone across a crowded town square. HTTPS and other cryptography protocols rely on a suite of algorithms that utilize properties of abstract algebra to encrypt data and enable secure communication across the internet.

Breaking a cryptographic algorithm like RSA or DH through brute force would take longer than a lifetime. However, these algorithms are not simply theoretical models, they must be physically coded and installed in systems in order for them to be of any use. This added dimension of physical implementation creates opportunities for hackers to extract information via what are known as side channel attacks.

"Side channel attacks" describes a large swatch of attack protocols that vary dramatically in implementation. However, what they have in common is their target: when software runs, it invariably leaves physical traces of its execution such as power consumption, computation time, and changes to the internal state of the memory. These physical traces are correlated with the execution of the program and can be recorded and analyzed in order to extract sensitive data.

For this project, I dug into Paul Kocher's 1991 paper entitled *Timing Attacks on Implementations of Diffie Hellman RSA DSS and Other Systems*. This seminal paper in the field of side channel attacks established the theoretical feasibility of extracting sensitive information from externally detectable timing differences in the execution of popular encryption protocols such as RSA. First, I will explain key background information that will make the mechanisms of Kocher's attack more clear. Then I will go through the logic of his attack and findings from his paper. Next I will detail

my attempt to recreate Kocher's attack. Lastly, I will discuss some of the next steps I would have explored if I'd had more time this semester.

# 2 Background

## 2.1 Public Key Cryptography

Public key cryptography is a class of cryptographic protocols which allows two parties to share secret information over an open channel without ever having exchanged information beforehand. Prior to the invention of Public Key Cryptography in the 70s, if Alice and Bob wanted to share secret information in a public forum, they had to do so using symmetric cryptography. The 'symmetric' part refers to the fact that Alice and Bob would both have to possess the same information in order to exchange their secret. This information would need to be exchanged in person to be truly secure.

Public key, or asymmetric cryptography, allows Alice and Bob to exchange a secret without ever having met in person. It utilizes what's called a 'trap-door function', which describes a mathematical function that is easy to compute in one direction, but very difficult to invert.

Kocher's paper discusses an attack focused on breaking RSA and Diffie Hellman, two standard (albeit at this point slightly outdated) public key encryption techniques. In order to understand how Kocher proposes attacking these systems, it's helpful to have a broad understanding of how they work. We'll use Diffie-Hellman as our model example. Diffie Hellman works as follows:

First, you choose a large prime number $P$, call this your key. Then you choose a generator $g$ of the multiplicative group $Z_p$. Then Alice and Bob both choose an integer, $a$ and $b$, and these are their private keys.

Alice and Bob will each calculate their own public key as follows: Alice: public key $A \equiv g^a \pmod{P}$ Bob: public key $B \equiv g^b \pmod{P}$

Then Alice and Bob both share their public keys with one another (it doesn't matter if anyone overhears and copies down one of their keys, that's why they're public!)

Then Alice and Bob compute a shared secret as follows:

$$\text{Alice computes: } B^a \pmod{P}$$

$$\text{Bob computes: } A^b \pmod{P}$$

Which works out to $g^a b \pmod{P}$, the same shared secret!

Bob and Alice can both quickly calculate $g^a b \pmod{P}$ with the information they have, however, in order to calculate their shared secret, Eve would have to extract either $a$ from $A$ or $b$ from $B$.

| Element | Purpose | Known By? |
|---|---|---|
| $P$ | Prime modulus | Public |
| $g$ | Generator | Public |
| $a$ | Alice's private key | Alice |
| $b$ | Bob's private key | Bob |
| $A = g^a \bmod P$ | Alice's public key | Public |
| $B = g^b \bmod P$ | Bob's public key | Public |
| $S = A^b \bmod P = B^a \bmod P$ | Shared secret | Alice and Bob |

This turns out to be an extremely hard problem, especially as the modulus $P$ gets larger. This is called the discrete log problem, and using cryptographically secure moduli (modern standard is 2048 bits), solving the discrete log problem would take longer than a lifetime.

## 2.2 Fast Modular Powers

Now, a trapdoor function has to both make calculating $a$ from $A$ or $b$ from $B$ (solving the DLP) take a long time, but it also needs to make calculating $g^{ab}$ or $g^{ba}$ quick for Alice and Bob. I glossed over that above, but take a second to think about it. In order for the algorithm to be cryptographically secure, you would need to use extremely large numbers. Calculating

$$183294023482034 98^{098364107650183947} \bmod 23948029384029384923048$$

using a standard exponentiation and reduction function would be quite slow. If this was a truly cryptographically secure system the exponent would be at least 2048 bits. That means the exponent would be approximately $2^{2048}$, and performing that many multiplications would take a modern computer billions of years.

Thus, there is a more popular implementation that is used to calculate modular powers called the "square and multiply" method. Square and multiply utilizes the fact that: if you square $a^n \pmod m$, you can get $a^2 n \pmod m$ in one multiplication instead of in $n$. It works as follows:

- Convert the exponent $b$ into its binary form.
- Process the binary digits one by one (from either direction).
- For every digit in the binary representation, square the current result. If the binary digit is 1, additionally multiply the result by the base. (This follows from the fact that $a^{2k+1} = a \cdot (a^k)^2$).
- After every multiplication or squaring step, compute a modular reduction to keep the numbers small and manageable.

This optimization makes algorithms like Diffie Hellman feasible, however it also introduces input based timing variations that, when detected, can leak information such as Bob or Alice's secret key! This is where side channel timing attacks come into play.

## 2.3 Timing Attacks

As previously mentioned, timing based attacks are a type of side channel attack that extract secret data by measuring and evaluating the runtime of an algorithm. Here's where we will take a moment to slow down and appreciate how difficult it is to do the *measuring* step of a timing attack. If Eve is launching a timing based attack on Bob and Alice, she will need accurate timing measurements of their function in order to proceed. Putting aside the issue of access and permissions for a moment, even assuming that Eve had the ability to hit "start" on a highly granular timer right when Bob began calculating his public key and "end" right as the function finished, there are a lot of factors during the execution of a program which are independent of the algorithm itself that can impact the amount of time it takes to finish running. For example, cache behavior, OS interrupts, core switching, etc. are all more or less unrelated to the algorithm, but will show up in Eve's timing measurements and make her data more noisy. Thus, timing attacks often become signal detection problems.

Figuring out how to detect a meaningful signal (difference in function execution timing based on input a vs. b) from background noise (cache hits, OS interrupts, etc) introduces many new challenges to executing a side channel attack, and there are many different strategies for dealing with this signal detection problem based on the relative strength of the signal and the nature of the noise. For Kocher's timing attack, the relevant part to understand is what we expect the noise to look like. If the noise is approximately randomly and independently distributed across measurements, then averaging many measurements of the same computation will cause the measurement to converge to the signal, allowing the signal to propagate through the noise. This is the core statement of the Central Limit Theorem, that the mean of a large number of noisy observations converges toward the true underlying value as long as the noise is independent and randomly distributed. However, this also means that if the noise does not follow these requirements, for example if the noise is highly structured, e.g. every fifth run there's a OS interrupt, then averaging many runs will not converge cleanly and the signal may remain buried in noise. Kocher's attack assumes that timing noise is approximately randomly and independently distributed, which allows him to address the signal detection problem by taking many measurements and averaging them.

# 3 Kocher's Attack

## 3.1 Introduction and Intuition

In Paul Kocher's 1996 Paper "Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems," Kocher proposed a statistics based timing attack to extract information from public key encryption protocols such as those listed in the paper title.

The premise of the attack is that the attacker is trying to uncover a private key $x$ that is used in a public encryption scheme. Kocher outlines the following prerequisites to his attack:

- The victim uses a function to compute fast modular powers which runs in non-constant time and the attacker knows the design of the victim system.
- He assumes that the victim encrypts or decrypts many different values of $y$ using the secret key $x$,

  $$T = y^x \bmod n,$$

  where all the $y$ values, $n$ and computation time ($T$) for each $y$ is known to the attacker.
- The same secret exponent $x$ must be used for all of these encryption instances.

Kocher writes than as long as these constraints are met, the attack can be leveraged on any system. However, the example function he gives is an encryption protocol which leverages the standard bit-by-bit square and multiply function which is the default for many systems as it can compute modular exponentiations exponentially faster than the brute force method. The attack is best understood if you assume that the victim utilizes some variant of this algorithm.

**Algorithm 1** Fast Modular Powers

```
Let s ← 1
for k = 0 to w do
  if bit k of x is 1 then
    R_k ← s · y mod n
  else
    R_k ← s
  end if
  s ← R_k mod n
end for
return R_w
```

This attack is both a side-channel timing attack and a signal detection problem. Timing analysis is used to extract the signal, which can then be analyzed to extract sensitive information. Note that in the fast modular powers algorithm detailed above, you only perform a multiplication if the

current bit is 1, otherwise you skip the multiply step. This conditional execution of the multiply step is the signal we want to detect. This signal is leaked by variation in the cost of a single multiply step. Given a particular base and intermediate value, the multiply step may require only one or zero modular reductions, or it may trigger many modular reductions. Over many known inputs, the small timing differences triggered by the presence or absence of the multiply step can become statistically visible.

To illustrate how runtime analysis can be used to extract this signal, Kocher provides an extreme hypothetical. Consider a scenario where the target system uses a modular multiplication function which is usually very efficient, but for some base texts $y$ and intermediate result values $s_b$, the multiply calculation takes a long time. In this extreme case, assume that slow multiply steps take longer than an entire single modular exponentiation usually takes. Then, if the total modular exponentiation time for a given intermediate result $s_b$ is fast, we know that the very long multiplication step couldn't have occurred, so the $bth$ exponent must not be set, and vice versa if the total modular exponentiation time is very slow.

### 3.2 Attack Roadmap

The attacker begins with a dataset $D$ of $n$ ciphertexts $c_j$ and their observed execution times $T_j$ for the modular exponentiation

$$c_j^x \bmod N,$$

where $x$ is the secret exponent of length $w$ bits and $N$ is the public modulus.

Now, the information we *really* want is what the value of the exponent is, bit by bit. If we had measurements of how long the square and multiply step took for each bit of the exponent for some $c_j^x \bmod N$ then determining the exponent bit by bit would be rather simple. We could go through each timing measurement and when the timing measurement was longer than the average, we could conclude that the bit was set, since a set bit triggers an additional multiplication step. However, under the premises of this attack we don't have bit by bit timing measurements. What we need is some way to make meaning of the set of overall timing measurements, $T_j$, that we have. This is where Kocher gets creative, using statistical tools to detect the signal from these overall timing measurements.

The first key thing to recognize is that the amount of time added to an overall $T_j$ from one multiply step is not the same across all ciphertexts. This harkens back to the previously mentioned idea that sometimes the multiplication calculation may be fast, and sometimes it may be slow, depending on the value of the base right before the multiplication step happens.

Here's a toy example to illustrate this point:

$$\textbf{Case 1 (Slow Multiply):} \qquad b = 10, \quad p = 17, \quad \text{exponent} = 30$$

$$\textbf{Case 2 (Fast Multiply):} \qquad a = 8, \quad p = 17, \quad \text{exponent} = 30$$

If we calculate

$$a^{30} \bmod p \quad \text{and} \quad b^{30} \bmod p$$

using a basic bit-by-bit square-and-multiply algorithm, then when processing the third set bit, the intermediate multiply step gives:

$$\textbf{Case 1:} \qquad 9 \cdot 10 \bmod 17 = 90 \bmod 17 = 5$$

To reduce 90 mod 17, we subtract 17 repeatedly:

$$90 - 5 \cdot 17 = 5.$$

Thus, we subtract 5 copies of 17, which we count as 5 operations.

$$\textbf{Case 2:} \qquad 4 \cdot 8 \bmod 17 = 32 \bmod 17 = 15$$

To reduce 32 mod 17, we subtract 17 once:

$$32 - 1 \cdot 17 = 15.$$

Now, we can leverage the fact that the cost of a multiply step is variable to expose the exponent bit by bit. Kocher's claim is that we can detect a set bit by guessing the first b bits of the exponent, calculating intermediate timing measurements for each ciphertext under that guess, and subtracting those predicted timings from the full measurements. If our timing guess is correct, then we are removing a structured, input-dependent component from each timing measurement, reducing the variance since we removed a true source of timing differences. If our guess is incorrect, we are subtracting essentially random amounts from each timing measurement, injecting noise and increasing the spread. This is the crux of Kocher's attack: using variance to determine the exponent bit by bit.

Consider some exponent $e$ and pick a single bit position $b$. Suppose that when $b = 1$, the multiply

step it adds is either 1 microsecond or 10 microseconds (depending on what the intermediate operand is). When $b = 0$, there is no multiply at that step. If $b = 1$ and you correctly guess that $b = 1$, then you calculate the time added to the exponentiation for each cipher text based on $b = 1$ and subtract either 10 microseconds or 1 microseconds from the total time, removing 1 or 10 microseconds from the total computation time that did truly exist. However, if you subtract these 10 or 1 microseconds from the total time and $b = 0$, then you're introducing a step that never happened, and so the residuals you compute will have a missing 10 or 1 microseconds in inconsistent ways.

That hopefully provides some intuition for why Kocher uses Variance as the tool to statistically detect a 0 or 1 bit. Below is a slightly more fleshed out mathematical explanation of the strategy just described.

Kocher models each observed timing as the sum of three parts: a contribution from the exponentiation steps, a measurement error term, and other small sources of overhead. For a given sample $j$, we can write

$$T_j = e_j + \sum_{i=0}^{w-1} t_{j,i},$$

where $t_{j,i}$ is the timing contribution associated with bit position $i$ of the exponent, and $e_j$ captures noise such as loop overhead and measurement error.

To study a guess for the first $b < w$ bits of the exponent, it is useful to split the total timing into a prefix and a suffix:

$$T_j = e_j + \sum_{i=0}^{b-1} t_{j,i} + \sum_{i=b}^{w-1} t_{j,i}.$$

Define

$$P_j = \sum_{i=0}^{b-1} t_{j,i} \qquad \text{and} \qquad S_j = \sum_{i=b}^{w-1} t_{j,i},$$

so that

$$T_j = e_j + P_j + S_j.$$

The first simplifying assumption Kocher makes is that the timing contribution from bit $i$ is approximately independent of the timing contribution from bit $i+1$. This may not be literally true in every implementation, since the result of one modular multiplication can affect the size of the next intermediate value. But for the attack, the dependence is weak enough to treat the bit contributions as approximately independent.

The second simplifying assumption is that the error term $e_j$ is drawn from the same noise distribution for all samples. In practice, individual measurements can be distorted by interruptions such as cache hits/misses, context switches, or other operating-system activity. Still, the attack only

needs the noise to be roughly stable across the dataset.

Under these assumptions, variance is the right statistic to examine because independent sources of variability add:

$$Var(T_j) = Var(e_j) + Var(P_j) + Var(S_j).$$

If the noise term is small or approximately constant, then

$$Var(T_j) \approx Var(P_j) + Var(S_j).$$

Now suppose our guess for the first $b$ bits is correct. Then the computed prefix matches the true prefix, so subtracting it from the measured time leaves only the suffix:

$$T_j - P_j = e_j + S_j.$$

The variance of this residual is therefore just the variance of the remaining suffix plus noise:

$$Var(T_j - P_j) \approx Var(S_j) + Var(e_j).$$

If our guess for the first $b$ bits is wrong, then the computed prefix differs from the true prefix by some mismatch term $\Delta_j$. In that case,

$$T_j - P_j = e_j + S_j + \Delta_j,$$

so the residual contains extra variability:

$$Var(T_j - P_j) \approx Var(S_j) + Var(e_j) + Var(\Delta_j).$$

Since $Var(\Delta_j) > 0$, the residual variance is larger when the guessed prefix is incorrect. This is the key test in Kocher's attack: the correct prefix is the one that makes the residual timings as low-variance as possible.

### 3.3 Experimental Results

While Kocher did not actually run his theorized experiment, he did take some timing measurements in order to verify the statistical foundations of his attack framework. Specifically, he demonstrated that modular exponentiation times follow a normal distribution across different inputs, which is a necessary precondition for the statistical attack to work as indicated in the background section on

timing attacks. The measurements following a normal distribution is a good indication that the noise is not highly structured, and therefore provides indications that the CLT may work in order to recover the signal.

### 3.4 Defenses: CRT and Montgomery Multiplication

Kocher also briefly touches on techniques that could potentially obfuscate the signal. He mentions using the Chinese Remainder Theorem (CRT) and Montgomery multiplication, which are both modular arithmetic tools that could either complicate the timing signal or reduce it in computing fast modular powers. This was not the focus of my experimentation, so I didn't spend much time reading about these countermeasures in detail. However, using algebraic-based and computationally-grounded security measures have been explored in greater detail and remain an interesting area of research in combatting side channel timing attacks.

## 4 Recreating the Attack

I endeavoured to recreate Kocher's attack for two main reasons, the first of which being that side channel memory attacks are messy and detailed, and this seemed like a good way to embrace and understand the detail of Kocher's attack. The second reason is that Kocher's paper is old and highly theoretical, and I was curious if his attack was still applicable in the context of modern systems.

The final re-creation code can be found here: Click here

The code was written in C and executed on a Linux operating system. The setup of the recreation was as follows:

- I wrote a square-and-multiply function that runs in non-constant time.
- The attack code first uses a modulus $P$ and enciphering exponent $e$ to encipher randomly generated ciphertexts. I time these calculations and store the timing measurements in a table with their corresponding base texts. These are the $T_j$ measurements that the attacker would theoretically "overhear" and utilize for their attack.
- I then ran these baseline timing measurements through the "compute bit" function. This is where I put Kocher's theories into practice, attempting to calculate intermediate timing measurements with exponent guesses and detect the correct guess using variance.
- Finally, I printed the difference between the true exponent ($e$) and the exponent that my testing guessed. I was ultimately only able to guess the exponent with 50% accuracy, which is essentially random.

Now, I will walk through why some of the key design choices that you see in the Git were made, and then discuss some hypotheses I have about why the attack was ultimately unsuccessful.

#### 4.0.1 Design choice 1: 64 bit prime modulus and 32 bit exponent

When I wrote my first draft of the attack code, I used 512 bit modulus and key. 512 was the standard cryptographically secure length for an RSA modulus until the late 90s, so part of the reason why I chose such large numbers was to try and more accurately simulate the functionality and timing of a modern cryptographic modular exponentiation operation. Additionally, I hypothesized that using a larger modulus (and therefore larger bases) and a larger exponent would increase the amount of time that any given operation took, therefore make the timing differences more detectable. However, it also seemed plausible that increasing the length of each operation would accumulate more noise and therefore drown out the signal (the signal being the value of a given bit b).

Thus, I ran a small experiment to investigate the relationship between signal strength and argument sizes. For each combination of modulus size and exponent size, I generated a random base within the range [0...modulus-1], I generated a random prime modulus of the appropriate length, then constructed two exponents of length e: one of the form $e1 = 10000...$ and one of the form $e2 = 11111....$ I timed $b^{e1} \bmod P$ and $b^{e2} \bmod P$ across many measurements and averaged the results. To quantify signal strength, I computed a signal-to-noise ratio: the absolute difference in group means divided by the larger of the two standard deviations.

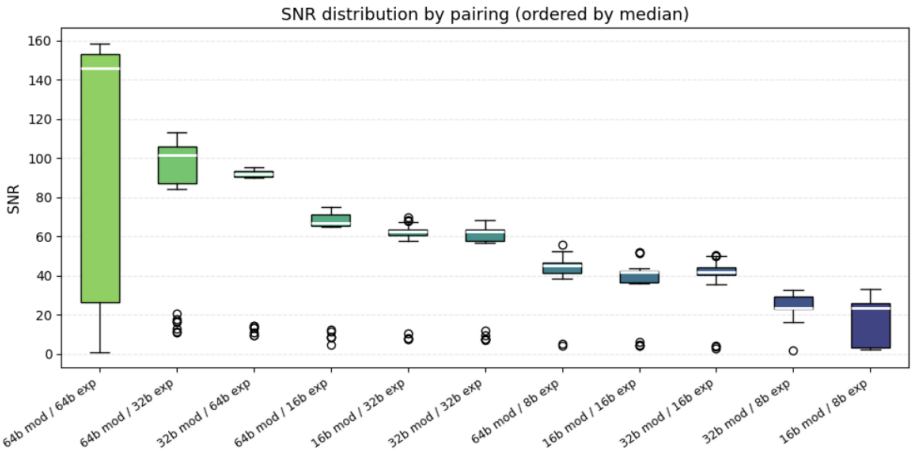



Figure 1: SNR for different mod, exponent sizes

The results from 30 trials per combination are shown above. Based on this data, I ultimately decided to go with a 64 bit modulus and 32 bit exponent, since the median SNR for this combination

was high relative to most of the other possible pairings and there was relatively low variation. While $64x64$ had the highest median SNR by quite a lot, it also had a very large spread which gave me less confidence in the reliability of the SNR. Thus, I elected to go with the 64 bit modulus and 32 bit exponent. More specifically, for all of the timings in my experiment I used: $e = 4294967291$, $mod = 18446744073709551557$.

#### 4.0.2 Design choice 2: Baseline timing measurement collection

The next important piece of the attack that I considered was how to collect the best possible baseline timing measurements of random ciphertexts, as these timings would be the foundation of the statistical attack. The first step in this process was determining how to actually generate the random base texts.

I used a native C pseudorandom number generator, which generated a random number with equal likelihood for all numbers in the range $\{0, \ldots, \text{mod} - 1\}$. Note that this is not the same as an equal distribution across all possible bit lengths, since there are exponentially more numbers with larger bit lengths (e.g. $2^{15}$ possibilities vs. $2^2$). I had a hard time discerning from Kocher's paper what size bases he used or imagined using in this attack, but this method produces a distribution of numbers mostly of length 64 bits. Using similarly sized base values seemed like a safe choice, so as not to introduce unnecessary variation.

Before taking the actual timing measurements, I did 3000 "Warmup" measurements, which used the same modulus and exponent as the actual baseline timings would, called the same timing function, and generated random ciphers in the same way. I decided to add this warmup period after experimentally determining that a warmup was required to reduce the impact of cache warming on the data– initially there was a coefficient of variation among the baseline timings of approximately 305%. However introducing a warmup brought this down significantly, and 3000x warmpup seemed to be the sweet spot that got rid of cache-warming induced variability in the data while also not being too computationally taxing. Next, I generated n base texts randomly as described above (n was a command line argument, which I generally kept in the range of 1,000-2,000). For each of these n cipher texts b, I followed the following procedure:

1. Timed $b^e$ mod $m$ 5000 times.
2. Percentile filtered the top ninety nine percent and then averaged those results to store as the timing measurement for ciphertext $b$.

Then for each batch of $n$ ciphertexts and corresponding measurements, I sorted them by time and kept the middle ninety nine percent, to try and reduce the impact of outliers.

To take the measurements, I used the timed square and multiply function that I wrote. This included fencing before every call, to minimize the impact of out of order execution on the timing measurements. I ended up using get process cpu time as the actual timing mechanism, which I arrived at after testing out several different timing libraries. Ultimately, get process cpu time seemed to be both highly granular (accurate on the scale of +-20 clock cycles), and it measures wall clock timing–which is the time that the function is actively running, excluding time taken for things like OS interrupts. This makes the timing much more replicable, which is important for the later steps in compute bit when we want to accurately recreate the baseline timing measurements.

The square and multiply function that I wrote can be seen in more detail in the git, but these are the major relevant details of its construction.

#### 4.0.3 Design choice 3: Compute bit design

After collecting the baseline timing measurements, the core of the attack is the compute bit procedure, which recovers the target exponent one bit at a time using the method proposed by Kocher.

The function begins with the same 3000-operation warmup used during baseline collection. This is done to prevent timing variation due to the cache being cold at the start of the procedure, and to ensure that the timing environment during the attack matches the environment during baseline collection as closely as possible.

The bit-recovery loop maintains a running partial exponent `curr_exp`, initialized to 1 to reflect the fact that the most significant bit (MSB) of a fixed-width exponent is assumed to be 1. At each iteration, the algorithm extends `curr_exp` by one bit by constructing two candidate exponents:

$$\texttt{guess0} = \texttt{curr_exp} \ll 1, \qquad \texttt{guess1} = (\texttt{curr_exp} \ll 1) + 1.$$

These correspond to appending a 0-bit or a 1-bit respectively. The goal at each step is to determine which candidate matches the true exponent, using timing consistency with the baseline measurements.

For each ciphertext in the baseline dataset, the attack collects 5000 timing measurements of $b^{\texttt{guess0}} \bmod M$ and 5000 measurements of $b^{\texttt{guess1}} \bmod M$. An `lfence` barrier is executed before each call to `timed_square_multiply` to reduce the effects of out of order execution, as previously mentioned.

The resulting 5000 raw measurements for each guess are then filtered and averaged using median absolute deviation (MAD) filtering with a sensitivity threshold of 3. This step is necessary because timing distributions are typically not normal, rather they are right-skewed, consisting of a dense

cluster of fast executions near the true runtime and a long tail of slower executions likely caused by cache misses, branch mispredictions, and operating system interrupts.

The mean is highly sensitive to these outliers, whereas the median is robust. MAD filtering allows us to remove measurements that are further than three standard deviations from the median, which produces a stable central estimate that removes timing spikes while preserving the underlying distribution of valid execution times.

To reduce systematic bias from cache state, the order in which guess0 and guess1 are measured was alternated across ciphertexts: for even-indexed ciphertexts, guess0 is measured first; for odd-indexed ones, guess1 is measured first. Without this, whichever guess is always measured second benefits from a warmer cache, which would introduce a consistent directional error into the timing comparison. (This was a design suggestion suggested by Martín Ochoa in his Kocher recreation accessible here: https://blog.zksecurity.xyz/posts/timing-kocher/)

Once filtered timings are collected for both guesses across all ciphertexts, a residual is computed for each ciphertext and each guess: the difference between the baseline timing and the guess timing. The intuition is that if guess0 is the correct partial exponent, it executes the same sequence of squarings and multiplications as the true exponent up to this bit, so its timing should closely track the baseline. The incorrect guess diverges from the true computation path at this bit position and will produce residuals with higher variance. Thus the correct guess should yield tighter residuals.

Before computing variance, a delta filter is applied. This was a strategy recommend by Kocher in his original paper, and once again implemented by Ochoa in his recreation. For each ciphertext, the absolute difference between its guess0 and guess1 timings is computed, and only ciphertexts in the top 95% by this delta are retained. Ciphertexts with small deltas between the two guesses carry little discriminating information, the two exponents happen to take similar amounts of time for that base, so including them adds noise without adding signal. Skewing toward high-delta ciphertexts sharpens the variance comparison.

Variance is then computed over the retained residuals for each guess. The guess with lower residual variance is selected as the correct bit, curr exp is updated accordingly, and the loop moves to the next bit.

#### 4.0.4 Experimental results

The attack yielded 15 correct bits and 17 incorrect bits for a majority of the runs, which indicates an essentially random probability of guessing the correct bit. Also importantly, as Kocher notes in his paper, after an incorrect bit is guessed once, every bit guessed after that is random. The first incorrect bit in the 20 trials I ran appeared within the first 3-4 bits of the guessed exponent,

confirming that a majority of the correctly guessed bits were random guesses.

There are several reasons why this recreation attempt could have failed. This includes logical failures in the experimental design, errors with the timing mechanisms used to collect the baseline or intermediate data, or a structural flaw with the premise such as a very weak signal or noise which is not randomly uniformly distributed .

While any of these issues may have been present, the one which seemed the most important to evaluate first was the presence of the signal, since if there is no signal there is no way to execute the test. In order to evaluate the presence of the signal I added a "dummy delay" function to my timed square and multiply. Dummy delay took in a number, the size of the delay, and then allocated and filled an array of that size before doing a multiplication step. I then timed a single bit exponentiation step for bit = 0 and bit = 1 for a constant base and modulus. My findings are in the table below:

| **delay_size** | **bit1 mean ± std (ns)** | **bit0 mean ± std (ns)** | **SNR** |
|---|---|---|---|
| 0 | $1744.8 \pm 143.3$ | $1707.0 \pm 138.7$ | 0.2634 |
| 10 | $2056.5 \pm 8.4$ | $1998.7 \pm 8.3$ | 6.8491 |
| 50 | $3750.3 \pm 24.0$ | $3566.4 \pm 14.5$ | 7.6607 |
| 100 | $6193.0 \pm 27.9$ | $5884.5 \pm 28.3$ | 10.8869 |
| 200 | $10504.6 \pm 38.5$ | $9935.4 \pm 43.2$ | 13.1795 |
| 500 | $23428.4 \pm 74.2$ | $22112.5 \pm 66.0$ | 17.7433 |
| 1000 | $44956.6 \pm 89.7$ | $42367.7 \pm 86.0$ | 28.8567 |

Table 1: Timing measurements and signal-to-noise ratio for different delay sizes

These initial results indicated that without false inflation, the signal was drowned out by the noise. However, the SNR was calculated using rudimentary means, so I wanted to dig a little bit deeper and see if a more advanced statistical investigation may yield a detectable signal.

Therefore I applied three two sample hypotheses tests: Welch's t-test, Mann-Whitney U, and the Kolmogorov-Smirnov test, to the bit=1 and bit=0 timing distributions at each delay size in order to determine if there is a true signal within the timing measurements. These tests are designed to examine two sets of measurements and determine if points in both distributions could have plausibly been drawn from the same underlying distribution. A p-value at or near zero means the answer is no, the two distributions are statistically distinguishable. Critically, even at delay size zero, where no artificial delay was inserted, all three tests returned p-values of effectively zero (e.g. t-test p = 8.72e-69), meaning the single extra modular multiply performed on a 1-bit versus a 0-bit produces a timing difference that is statistically detectable with 10,000 samples, confirming that a real timing signal exists even without any amplification.

From these results, it seems like the non-constant time square and multiply function does leak timing information, but the signal is so weak that you need thousands of samples to see it even in

the very controlled setting of the experiment described above. This indicates that while the signal Kocher describes detecting theoretically exists, it would likely require much greater computational power to reliably detect.

## 4.1 Next steps

Now that I've confirmed that a signal does exist, I would experiment with changing the number of repetitions of timing measurements for a single cipher text (baseline as well as intermediate measurements), and the number of total cipher texts that would allow for effective signal propagation. I think this would be more feasible if I ran the code on, for example, Frostbyte, since the sensitivity of the signal indicates that a large number of trials at every level of measurement would be needed, and I found that doing more than 10,000 ciphertexts and 500 reps per ciphertext was computationally infeasible on my computer in a realistic time frame.

Another piece of this experiment that I wish I had more time to dig into is the structure and nature of the noise observed in the timing measurements. While I was able to confirm that my baseline timing measurements fell into a normal timing distribution for any given cipher text, and thus theoretically taking many samples and averaging them should have reliably exposed a signal, I think it would be interesting to try and isolate the different sources of noise at play and their precise contributions to the overall timing measurement, and this could potentially pave the way for a cleaner signal measurement which would reduce the number of samples needed to detect the signal.

Additionally, I've identified several papers that build off of the core theoretical attack that Kocher outlined in his paper, but seek to apply his theoretical framework in the context of a more realistic problem. One of the pieces of Kocher's attack that was unsatisfying is the amount of information the hacker needs to have access to in order to conduct the attack. This makes it feel like much more of a theoretical or academic exercise rather than an evaluation of true security gaps. A practical implementation of the timing attack (Dhem et al 1998) and Timing attacks are realistic (Stanford 2019) are two background papers which adapt Kocher's theoretical framework in the context of a more realistic security gap. Digging into the different premises established in these papers, as well as their attack strategies and findings would provide an interesting next step allowing me to relate my understanding of this problem to more realistic systems.

## 4.2 Sources

National Institute of Standards and Technology. Physical Security Testing Workshop: Papers. National Institute of Standards and Technology, https://csrc.nist.gov/csrc/media/events/physical-

security-testing-workshop/documents/papers/physecpaper19.pdf. Accessed 14 May 2026.

Ochoa, Martín. "Kocher's Timing Attack: A Journey from Theory to Practice." ZK Security Blog, 19 Sept. 2025, https://blog.zksecurity.xyz/posts/timing-kocher/. Accessed 14 May 2026.

Kocher, Paul C. Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems. 1996, https://paulkocher.com/doc/TimingAttacks.pdf. Accessed 14 May 2026.